A RULE-BASED COMPILER FOR THE BANGLISH PROGRAMMING LANGUAGE WITH ENHANCED ERROR REPORTING

Student Information:

Name: Niloy Chowdhury
Roll: 2107117
Dept: Computer Science and Engineering

Course Information:

Course No. : CSE 3212
Course Title : Compiler Design Laboratory

Teacher Information:

Mr. Md. Badiuzzaman Shuvo
Lecturer
Department of Computer Science and Engineering
Khulna University of Engineering and Technology,Khulna

Ms. Subah Nawar
Lecturer
Department of Computer Science and Engineering
Khulna University of Engineering and Technology,Khulna

Abstract

This report documents the complete design and implementation of a Banglish (Bangla written using English letters) programming language compiler. The project was developed as a lab assignment and evolved from an initial proposal into a fully functional educational compiler pipeline. Implemented in C, Flex, and Bison, the compiler performs lexical analysis, syntax analysis, abstract syntax tree (AST) construction, semantic analysis, symbol table management, intermediate code generation, and source-to-source translation from Banglish to C. The generated C code can be compiled and executed automatically, with runtime output saved to report files. The system introduces unique features such as Banglish-friendly error messages, static code quality analysis, mode-based execution, and partial compile-time evaluation, making it a comprehensive educational tool for understanding compiler principles.

1. Introduction

Compiler design is a fundamental area of computer science focused on translating high-level programming languages into lower-level representations while ensuring correctness and efficiency. Most modern compilers are designed for English-based programming languages such as C, C++, or Java. However, language localization and domain-specific languages (DSLs) are gaining importance for education and accessibility.

This project designs and implements a Banglish programming language compiler that allows programmers to write code using Banglish keywords while preserving standard compiler behavior. The motivation is to explore compiler construction techniques while extending traditional compilers with semantic intelligence, localized error reporting, and static program analysis. The compiler is implemented entirely using rule-based techniques in C with Flex and Bison, ensuring determinism and academic correctness.

What began as a proposal for a front-end compiler pipeline has since evolved into a full end-to-end system capable of lexing, parsing, analyzing, generating, compiling, and executing Banglish programs automatically.

2. Objectives

The main objectives of this project are:

1. To design a well-defined Banglish programming language syntax.
2. To implement a complete compiler using C, Flex, and Bison.
3. To perform lexical, syntactic, and semantic analysis.
4. To build an Abstract Syntax Tree (AST) for all major language constructs.
5. To generate intermediate representation using three-address code and source-to-source C translation.
6. To detect semantic errors and report them in Banglish with clear explanations.
7. To perform static code quality analysis such as unused variable detection.
8. To apply basic compile-time optimizations such as constant folding.

9. To support mode-based execution: analysis only, C code generation, and full execution.
10. To automatically compile generated C code and capture runtime output.

3. Overview of the Banglish Programming Language

3.1 Language Design

The Banglish programming language is a domain-specific language where keywords are written in Banglish, while operators remain symbolic to simplify parsing. It supports a rich set of constructs including variable declarations, control flow, loops, functions, and I/O statements.

3.2 Keyword Mapping

The following table shows the complete set of Banglish keywords supported in the current implementation:

Banglish Keyword	C / English Equivalent
purno	int
dosomik	float
torkik	bool
shunno	void
dhrubo	const
jodi	if
nahole	else
jotokhon	while
ghuro	for
koro	do

Banglish Keyword	C / English Equivalent
bachai	switch
khetre	case
onnothay	default
tham	break
chaliejao	continue
ferot	return
dekhao	print / printf
neo	input / scanf
kaj	function
shotti	true
mithya	false

3.3 Sample Program

Below is a sample Banglish program demonstrating variable declaration and conditional branching:

```
purno a = 10;
purno b = 20;

jodi (a < b) {
    dekhao(a + b);
}
```

4. System Architecture and Compilation Pipeline

The compiler follows a standard multi-phase architecture with seven distinct phases, each implemented as a separate module:

Phase	Module	Description
1	lexer.l	Lexical Analyzer — tokenizes source text and logs token stream
2	parser.y	Syntax Analyzer — validates grammar and constructs AST
3	ast.c / ast.h	AST Printer — outputs hierarchical program structure
4	semantic.c	Semantic Analyzer — performs meaning checks, reports errors/warnings
5	symbol_table.c	Symbol Table Dump — prints final symbol entries with scope
6	codegen.c	Code Generator — emits equivalent C source from AST
7	(auto)	C Compilation & Execution — compiles and runs generated C code

5. Lexical Analysis

Lexical analysis is performed using Flex. The lexer identifies tokens such as keywords, identifiers, literals, operators, and delimiters. Banglish keywords are mapped directly to token types. Line numbers are tracked to support precise error reporting. The lexer also handles lexical diagnostics for unterminated strings and comments.

Example token rule in lexer.l:

```
"purno"    { return INT; }
"jodi"     { return IF; }
"dekhao"   { return PRINT; }
```

6. Syntax Analysis

Syntax analysis is implemented using Bison. A context-free grammar defines the syntactic structure of the Banglish language. The parser validates program structure and constructs the abstract syntax tree. The grammar supports:

- Variable and constant declarations
- Assignments and compound assignments (+=, -=, *=, /=)
- Unary operations (++, --, !, unary -)
- If-else and else-if chains
- While, for, and do-while loops
- Switch-case-default statements
- Function definition and function calls
- Return, break, and continue statements
- Arithmetic, relational, and logical expressions with proper precedence

7. Abstract Syntax Tree (AST)

The AST represents the hierarchical structure of the source program. Each node corresponds to a language construct such as expressions, statements, or declarations. The AST enables further semantic analysis and code generation independent of the grammar.

AST node definitions, constructors, linked-list helpers, pretty-printer, and memory cleanup are handled in ast.h and ast.c.

7.1 Detailed AST Example

For the sample Banglish program shown in Section 3.3, the AST structure is:

```
Program
├── Declaration: purno a
│   └── Value: 10
├── Declaration: purno b
│   └── Value: 20
└── IfStatement
    ├── Condition: a < b
    │   ├── Identifier: a
    │   └── Identifier: b
    └── Body
        └── PrintStatement
            └── Expression: a + b
                ├── Identifier: a
                └── Identifier: b
```

7.2 Benefits of the AST

- ✓ Makes semantic checks straightforward.
- ✓ Enables type checking at expression nodes.
- ✓ Simplifies code generation for C source translation.
- ✓ Provides a visual representation for understanding program structure.

8. Semantic Analysis

Semantic analysis ensures that the program is logically correct. This phase uses a scoped symbol table to track identifiers, types, and scopes. The following checks are fully implemented:

Check	Description
Redeclaration	Detects redeclaration of the same identifier in the same scope
Undeclared usage	Reports use of identifiers that have not been declared
Constant assignment	Rejects assignment to constants after initialization
Constant without init	Flags constant declarations without an initial value
Void variable	Rejects void as a variable type
Break/continue context	Ensures break/continue only appear inside loops/switch
Return outside function	Detects return statements outside function bodies
Nested function decl.	Rejects function declarations inside other functions
Type mismatch	Warns on type mismatch in assignments and initializations
Uninitialized variable	Warns when a variable is used before being assigned
Division by zero	Detects literal zero denominators at compile time
Undeclared function call	Detects calls to functions that have not been declared

9. Banglish Error Reporting and Explanation

Unlike traditional compilers that provide brief cryptic error messages, this compiler produces descriptive Banglish messages with human-friendly explanations and suggestions. Error detection is rule-based and deterministic.

Example error message produced by the compiler:

> Error (line 4):
>
> 'purno' type er variable e dosomik value assign kora jabe na.
>
> Suggestion: Variable er type 'dosomik' kore nin.

This feature significantly improves readability and user understanding, especially for Bengali-speaking learners, without using any AI assistance.

10. Symbol Table Management

The symbol table module (symbol_table.h / symbol_table.c) maintains a scoped structure tracking all declared identifiers throughout the program. It supports:

- Scoped insert, lookup, and dump utilities
- Variable, constant, function, and parameter tracking
- Data-type helper functions for type inference and mismatch detection
- Final symbol table dump to output report

11. Static Code Quality Analysis

The compiler performs static analysis to detect potential issues that may not prevent compilation but affect code quality. These include:

- Unused variable detection
- Variables declared but never initialized
- Use of uninitialized variables
- Unreachable code patterns

Warnings are displayed in Banglish to assist programmers during development, consistent with the localized design philosophy.

12. Code Generation

The compiler generates equivalent C source code from a valid AST program. This source-to-source translation is handled by codegen.h and codegen.c, which traverse the AST and emit corresponding C constructs for all supported language features.

Example of generated C code from a Banglish program:

```
// Generated by Banglish Compiler
#include <stdio.h>
int main() {
    int a = 10;
    int b = 20;
    if (a < b) {
        printf("%d\n", a + b);
    }
    return 0;
}
```

13. Mode-Based Execution

A key addition beyond the original proposal is the mode-based execution system. The compiler supports three execution modes via command-line argument:

Mode	Command Argument	Description
Analysis Only	analysis or 1	Runs lexical + syntax + AST + semantic analysis only
Generate	generate or 2	Analysis + C source code generation
Execute	execute or 3	Analysis + C generation + C compilation + runtime execution

14. Build and Run Instructions

14.1 Build Commands

```
cd "D:/3-2/Compiler Project/Rule-Based-Banglish-Compiler-With-Enhanced-Error-Reporting"
unalias flex bison 2>/dev/null || true
flex lexer.l
bison -d parser.y
gcc lex.yy.c parser.tab.c ast.c semantic.c symbol_table.c codegen.c -o banglish.exe -lm
```

14.2 Run Examples

```
./banglish.exe demo_step1_lexical.bl demo_step1_lexical_report.txt analysis
./banglish.exe demo_step2_syntax.bl demo_step2_syntax_report.txt analysis
./banglish.exe demo_step3_ast_valid.bl demo_step3_ast_report.txt analysis
./banglish.exe demo_step4_semantic.bl demo_step4_semantic_report.txt analysis
./banglish.exe demo_step5_codegen_execute.bl demo_step5_pipeline_report.txt execute
```

15. Output Artifacts Produced

Depending on the mode selected, the compiler produces the following output files:

- Main report file (e.g., output.txt): tokens, summaries, AST, semantic diagnostics, symbol table.
- <source>_generated.c: generated C code from AST.
- <source>_generated.exe: compiled generated C binary (on successful compile).
- <source>_runtime_output.txt: runtime output of generated executable.
- <source>_c_translation_and_output.txt: combined generated C code and runtime output report.

16. Implementation Tools and Technologies

Tool / Technology	Purpose
C	Core implementation language for all compiler modules
Flex	Lexical analyzer generator
Bison	Parser and grammar specification
GCC	Compilation of generated C code and testing
Git / GitHub	Version control and project repository

17. Limitations

- ❖ The language remains a controlled subset for educational use; no full standard library support.
- ❖ No three-address code optimizer pass yet (planned extension).
- ❖ Function argument type/arity strict validation can be expanded further.
- ❖ Interactive runtime input handling is intentionally constrained during automated execute mode.
- ❖ Runtime execution is not embedded natively; it depends on GCC being available.

20. Future Work

Possible extensions of this project include:

- ✓ Full optimizer pass for three-address intermediate code.
- ✓ Code generation to native assembly.
- ✓ Advanced data-flow optimizations.
- ✓ Expanded standard library support (string handling, file I/O).
- ✓ Optional AI-assisted tutoring explanations integrated with the error messages.
- ✓ Interactive REPL (Read-Eval-Print Loop) for Banglish.

21. Conclusion

This project successfully demonstrates the design and implementation of a Banglish programming language compiler using Flex and Bison. What began as a proposed front-end compiler pipeline has evolved into a full end-to-end system capable of lexing, parsing, semantic analysis, AST construction, C code generation, compilation, and execution — all from Banglish source files.

By integrating localized syntax with advanced semantic analysis, static code quality checks, and source-to-source translation, the compiler extends traditional compilation

techniques while remaining academically sound. The project highlights how compiler principles can be adapted for localized and educational programming languages, and serves as a strong demonstration of the complete compiler design pipeline.

References

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. Compilers: Principles, Techniques, and Tools. Pearson.
2. Levine, J. Flex & Bison. O'Reilly Media.
3. Muchnick, S. S. Advanced Compiler Design and Implementation. Morgan Kaufmann.